

ScheduleSim — Final Documentation

Software Quality (CSS) Project — Phases 1–3

Team: Raul Tiron, Teodora Vlad, Cosmin Stan

University: UAIC

Language: Java 21 | **Build:** Maven 3.x

Table of Contents

1. [Project Overview](#)
 2. [Program Design and Implementation \(Phase 1\)](#)
 - 2.1 [Architecture](#)
 - 2.2 [Component Descriptions](#)
 - 2.3 [Algorithms](#)
 - 2.4 [Data Structures](#)
 - 2.5 [Process Lifecycle](#)
 - 2.6 [Input and Output Format](#)
 - 2.7 [Key Design Decisions](#)
 3. [User Manual](#)
 4. [Unit Testing \(Phase 2\)](#)
 - 4.1 [Testing Approach](#)
 - 4.2 [Test Suite Overview](#)
 - 4.3 [Mocking Strategy](#)
 - 4.4 [Code Coverage](#)
 - 4.5 [Defects Found](#)
 5. [Assertions \(Phase 3\)](#)
 - 5.1 [Assertion Strategy](#)
 - 5.2 [Assertions by Component](#)
 - 5.3 [Relationship to Defects Found](#)
 6. [Team Member Contributions](#)
-

1. Project Overview

ScheduleSim is a discrete event-driven simulator of process scheduling in an operating system. The simulator models a multi-processor system with limited RAM, virtual memory with disk swapping,

and a dedicated system process for handling system calls.

The simulator accepts a JSON configuration file describing hardware parameters and a list of user processes. It produces a complete execution trace as both a structured text file and a graphical Gantt chart rendered in a Swing window.

Technology stack:

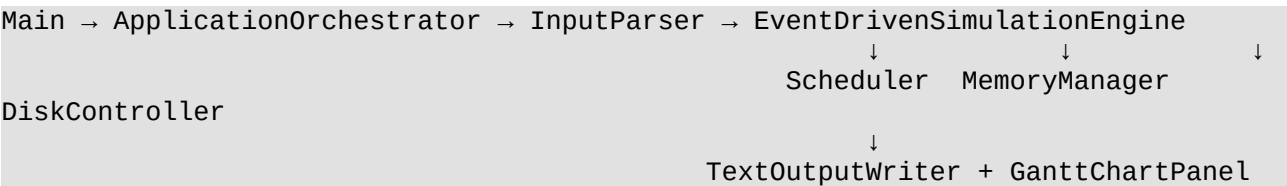
Tool	Purpose
Java 21	Implementation language
Maven 3.x	Build and dependency management
Jackson Databind 2.17.0	JSON parsing (only external logic dependency)
Java Swing	Graphical Gantt chart (only external UI dependency)
JUnit 5.10.2	Unit testing framework
Mockito 5.11.0	Mock objects for isolated testing
JaCoCo 0.8.12	Code coverage measurement

No external libraries were used for simulation logic, in compliance with the project specification.

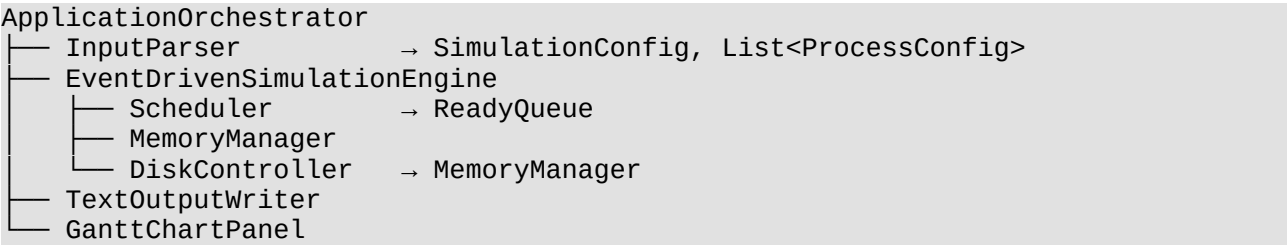
2. Program Design and Implementation (Phase 1)

2.1 Architecture

The simulator follows a clean pipeline architecture:



Component dependency graph:



The engine **orchestrates** all other components but does not duplicate their logic. Each component owns a specific slice of simulation state and makes decisions only within its domain.

2.2 Component Descriptions

Component	Package	Responsibility
Main	root	Entry point; reads input file path from command-line args
ApplicationOrchestrator	root	Wires all components; runs the full pipeline
InputParser	config	Deserializes JSON into <code>SimulationConfig</code> and <code>ProcessConfig</code> records
SimulationConfig	config	Immutable record of global simulation parameters with input validation
ProcessConfig	config	Immutable record of per-process configuration with input validation
EventDrivenSimulationEngine	engine	Core simulation loop; owns event queue, current time, syscall queue, log
Scheduler	scheduler	Round-Robin scheduling with processor affinity; owns the <code>ReadyQueue</code>
MemoryManager	memory	RAM accounting and LRU eviction planning
DiskController	memory	Sequential disk I/O operations; owns the disk timeline
Process	model.process	Mutable process entity with state, burst tracking, and last processor ID
Processor	model.simulation	CPU core state: current process and system-process occupancy
ReadyQueue	model.simulation	FIFO queue with predicate-based search
MinHeapPriorityQueue	util	Custom binary min-heap used as the event queue
TextOutputWriter	output	Structured text output (chronological, per-processor, disk ops)
GanttChartPanel	ui	Swing-based Gantt chart with color-coding and scrolling

2.3 Algorithms

Preemptive Round-Robin Scheduling

All user processes have equal priority and are served in FIFO order. When a process is dispatched onto a processor, two cases arise:

- `remainingBurstTime ≤ timeSlice` → A `BURST_COMPLETED` event is scheduled. The process runs to completion of the current burst.
- `remainingBurstTime > timeSlice` → A `TIME_SLICE_EXPIRED` event is scheduled. The process is preempted, `remainingBurstTime` is decremented by `timeSlice`, and the process returns to the tail of the ready queue.

Processor Affinity (Soft)

When multiple processors are free, the scheduler prefers the processor on which the process most recently executed (`lastProcessorId`). This is a soft preference — if the preferred processor is busy, any free processor is used. New processes (with `lastProcessorId == -1`) are assigned to the first free processor.

The system process has no affinity and uses any free processor.

LRU (Least Recently Used) Eviction

When a process must be loaded from disk but there is insufficient free RAM, the memory manager selects processes to evict using LRU:

1. Sort all loaded processes by `lastUsedTime` ascending (least recently used first).
2. Skip processes in `RUNNING` or `LOADING` state — they cannot be evicted.
3. Accumulate candidates until enough memory is freed.
4. Return the eviction plan as an `EvictionResult`.

"Last used time" is defined as the most recent timestamp at which the process was dispatched onto a processor.

Two-Phase Memory Reservation (Reserve / Commit)

To prevent double-booking of RAM while a disk transfer is in progress:

1. **`reserveSpace(amount)`** — Called when a disk load begins. Deducts from free memory immediately.
2. **`commitLoad(process, time)`** — Called when the disk transfer completes. Moves memory from reserved to used.

This ensures that a second disk load initiated while the first is in flight does not see the same free memory as available.

Event-Driven Simulation

The simulator uses a **discrete event-driven** model rather than a time-stepped approach. The engine

jumps directly to the next event's timestamp, skipping over idle periods. This is efficient because simulations may have long intervals (e.g., waiting for disk transfers, waiting for the system process period) between meaningful state changes.

Event priority ordering (lower value = processed first at the same timestamp):

Priority	Event Type	Rationale
1	PROCESS_RELEASE	New processes enter the ready queue before any scheduling at this timestamp
2	SYSTEM_PROCESS_COMPLETED	Frees a CPU before other completions so it is available for user processes
3	BURST_COMPLETED	Completion takes precedence over preemption at the same timestamp
4	TIME_SLICE_EXPIRED	Preemption processed after completions
5	SYSCALL_COMPLETED	Processes re-enter the ready queue after CPU-freeing events
6	SYSTEM_PROCESS_RELEASE	Periodic release assessed after CPU events settle
7	DISK_TRANSFER_COMPLETED	Disk completions add processes to the ready queue last

Two-Pass Scheduling (trySchedule)

Every time the simulation state changes, `trySchedule()` is called. It operates in two passes:

- **Pass 0 — System process priority:** If the system process is waiting and there are pending syscalls, it claims the first available processor.
- **Pass 1 — In-memory scheduling:** Matches ready processes already loaded in RAM with free processors. Multiple assignments can happen in one call.
- **Pass 2 — Disk load initiation:** If there are still ready processes not in memory, initiates exactly one disk load for the next such process.

Only one disk load is initiated per `trySchedule()` call because the disk is a single serial channel, and limiting to one load allows re-evaluation after each transfer completes.

System Process Scheduling

The system process is released strictly periodically at multiples of `systemProcessPeriod` (no drift). When released with pending syscalls and a free processor available, it executes all pending syscalls back-to-back without yielding (not subject to Round-Robin). If no processor is free at release time, it waits and gets higher priority over user processes when one becomes available.

2.4 Data Structures

Structure	Implementation	Usage
MinHeapPriorityQueue<Event>	Custom binary min-heap	Event queue; $O(\log n)$ add and poll
ReadyQueue	LinkedList-backed FIFO	Ready process queue with predicate search
LinkedHashMap<Process, Integer>	Java built-in	Loaded processes mapped to last-used time (LRU)
Queue<SystemCallRequest>	LinkedList-backed	Pending syscall queue (FIFO)
ArrayList<Processor>	Java built-in	CPU cores indexed by ID
ArrayList<ExecutionLogEntry>	Java built-in	Simulation execution log

The `MinHeapPriorityQueue` is a custom implementation as required by the specification (no external data structure libraries for simulation logic).

2.5 Process Lifecycle

```
NEW → READY → RUNNING → WAITING_SYSCALL → READY → RUNNING → ... → TERMINATED
      ↓
      LOADING (if not in memory) → READY (after disk load completes)
```

State	Invariant
NEW	Not in ready queue; not in memory
READY	In ready queue; may or may not be in memory
RUNNING	Exactly one processor has this process assigned; loaded in memory
WAITING_SYSCALL	Has a pending syscall in the syscall queue; not on any processor
LOADING	Disk transfer in progress; memory reserved but not yet committed; cannot be evicted
TERMINATED	All bursts completed; memory freed if was loaded

2.6 Input and Output Format

Input — JSON configuration file:

```
{
  "processors": 2,
  "memorySize": 100,
  "timeSlice": 4,
  "systemProcessPeriod": 20,
  "diskTransferRate": 10,
  "processes": [
```

```

{
  "name": "P1",
  "releaseTime": 0,
  "memoryRequired": 30,
  "executionSequence": [5, 2, 3, 4, 9, 4, 6]
}
]
}

```

The `executionSequence` is an odd-length array alternating between CPU burst durations and syscall durations. For `[5, 2, 3, 4, 9, 4, 6]`: 4 CPU bursts of lengths 5, 3, 9, 6 separated by 3 syscalls of durations 2, 4, 4.

Input validation (always active, throw `IllegalArgumentException`):

- All numeric parameters must be positive.
- The process list must not be null or empty.
- No process may require more memory than the total system memory (cross-validation).
- Execution sequences must be non-null, non-empty, and have an odd number of elements.
- All execution sequence values must be positive.

Output — text file (`output.txt`):

1. Chronological log — all events sorted by time.
2. Per-processor timeline — what each CPU did over time.
3. Disk operations — all saves and loads in order.

Output — Gantt chart (Swing window):

- One row per processor and one row for the disk.
- Color-coded per process: CPU bursts in a unique color, syscalls in a darker shade, disk loads in a semi-transparent shade.
- Scrollable panel for long simulations, with a legend.

2.7 Key Design Decisions

Decision	Rationale
Event-driven (not time-stepped)	Skips idle periods; $O(E)$ vs $O(T)$ complexity
Custom <code>MinHeapPriorityQueue</code>	Specification prohibits external logic libraries
Two-phase memory reservation	Prevents double-booking during concurrent disk transfers
Single disk channel (<code>diskBusyUntil</code>)	Models real serial disk; non-blocking for CPUs
One disk load per <code>trySchedule()</code>	Allows re-evaluation after each transfer; prevents queue flooding
System process runs uninterrupted	Matches specification; period is long enough for full completion

Decision	Rationale
Immutable records for value objects	<code>Event</code> , <code>SimulationConfig</code> , <code>ProcessConfig</code> , <code>ExecutionLogEntry</code> , etc. are Java records — concise, safe, auto-generated <code>equals/hashCode/toString</code>
Dependency injection for all components	Enables isolated unit testing with mock objects
Single-responsibility classes	<code>Scheduler</code> decides assignments, <code>MemoryManager</code> tracks memory, <code>DiskController</code> sequences I/O — no logic duplication

3. User Manual

Prerequisites

- Java 17 or later
- Apache Maven 3.x

Build and Run

Default input file (`src/main/resources/input.json`):

```
mvn clean compile exec:java -Dexec.mainClass="uaic.css.Main"
```

Custom input file:

```
mvn clean compile exec:java -Dexec.mainClass="uaic.css.Main"
-Dexec.args="path/to/input.json"
```

With assertions enabled (recommended during development):

```
mvn clean compile exec:java -Dexec.mainClass="uaic.css.Main" -Dexec.jvmArgs="-ea"
```

Output

- **Text output:** `output.txt` in the project root directory.
- **Gantt chart:** Opens automatically in a Swing window when the simulation completes.

Input File Format

Create a JSON file with the following structure:

Field	Type	Constraint	Description
<code>processors</code>	integer	> 0	Number of CPU cores
<code>memorySize</code>	integer	> 0	Total RAM (arbitrary units)

Field	Type	Constraint	Description
timeSlice	integer	> 0	Round-Robin quantum
systemProcessPeriod	integer	> 0	Period of system process release
diskTransferRate	integer	> 0	Memory units transferred per time unit
processes	array	non-empty	List of user processes

Each process entry:

Field	Type	Constraint	Description
name	string	non-empty	Process identifier
releaseTime	integer	≥ 0	Time at which the process is created
memoryRequired	integer	$> 0, \leq \text{memorySize}$	RAM footprint
executionSequence	array	odd length, all > 0	Alternating CPU bursts and syscall durations

If the input is invalid, the program terminates with an `IllegalArgumentException` describing the problem.

4. Unit Testing (Phase 2)

4.1 Testing Approach

Unit testing was applied to verify that each module handles its inputs correctly in isolation, including boundary conditions and invalid inputs. The goal was to discover whether a module can handle incorrect input data provided by other modules or by application I/O — not to fix errors at this stage.

Testing was performed with:

- **JUnit 5** — test framework and assertion library
- **Mockito** — mock object creation to simulate dependencies
- **JaCoCo** — code coverage measurement

To enable isolated testing, the `EventDrivenSimulationEngine` and `ApplicationOrchestrator` were refactored to support **dependency injection**: all major collaborators (`Scheduler`, `MemoryManager`, `DiskController`, `ReadyQueue`) are accepted as constructor parameters. This allows tests to inject mock objects instead of real components.

The same principle was applied to `TextOutputWriter`: a second method variant was added that

accepts any `java.io.Writer` (instead of a file path), allowing tests to use a `StringWriter` in memory rather than writing to disk.

To run all tests:

```
mvn test
```

To generate a code coverage report:

```
mvn clean test
# Report is generated at: target/site/jacoco/index.html
```

4.2 Test Suite Overview

Test Class	Component Under Test	Test Count	Key Scenarios Covered
ProcessTest	Process	30	Constructor validation, burst tracking, state transitions, <code>lastProcessorId</code> management, advancement to next burst
ProcessorTest	Processor	12	Assignment and release of user process, system process occupancy, invariant enforcement
ReadyQueueTest	ReadyQueue	21	Enqueue/dequeue, predicate-based search, FIFO ordering, null handling
MinHeapPriorityQueueTest	MinHeapPriorityQueue	15	Add, poll, peek, heap invariant preservation, single-element and empty cases
MemoryManagerTest	MemoryManager	30	Reserve/commit protocol, LRU eviction ordering, feasibility checks, class invariant
DiskControllerTest	DiskController	9	Disk timeline sequencing, eviction before load, correct event scheduling
SchedulerTest	Scheduler	18	Affinity matching, fallback to any free processor, in-memory vs. disk load pass, FIFO ordering
InputParserTest	InputParser	9	Valid JSON, missing

Test Class	Component Under Test	Test Count	Key Scenarios Covered
			fields, negative parameters, odd-length execution sequence, cross-validation
TextOutputWriterTest	TextOutputWriter	13	Chronological output, per-processor output, disk operations section, Writer-based output
EventDrivenSimulationEngineTest	EventDrivenSimulationEngine	20	Event dispatch, Round-Robin preemption, syscall flow, disk transfer flow, system process scheduling
Total		177	

4.3 Mocking Strategy

Mockito was used to replace real components with controlled doubles when testing a specific module in isolation. Examples:

- **SchedulerTest:** `MemoryManager` is mocked so `isLoading(process)` returns a controlled value, allowing the scheduler's two-pass logic to be tested without real memory state.
- **DiskControllerTest:** `MemoryManager` is mocked to return predictable eviction plans; the event queue is mocked to verify that `DISK_TRANSFER_COMPLETE` events are scheduled at the correct time.
- **EventDrivenSimulationEngineTest:** `Scheduler`, `MemoryManager`, and `DiskController` are injected as mocks, allowing individual event handler behaviors to be tested without triggering the full simulation pipeline.

This isolation means that a failing test points directly to the component under test, not to an interaction between multiple components.

4.4 Code Coverage

JaCoCo was configured to measure line and branch coverage across all production classes (the `ui` package containing `GanttChartPanel` was excluded from coverage requirements as GUI rendering is not amenable to unit testing). The coverage report is generated to `target/site/jacoco/index.html` after `mvn clean test`.

4.5 Defects Found

Three defects were discovered by tests written to expect correct behavior. The tests were written to describe the expected behavior first; when run against the existing code, they failed, confirming the defect.

Defect 1 — Integer overflow in system process scheduling

Field	Detail
Class	EventDrivenSimulationEngine
Method	handleSystemProcessRelease
Description	<code>nextSystemProcessReleaseTime</code> += <code>config.systemProcessPeriod()</code> accumulates into a 32-bit <code>int</code> . For simulations with a large period or long runtime, the value can overflow and become negative.
Expected	The engine guards against overflow (e.g., stops scheduling when time would exceed <code>Integer.MAX_VALUE</code>).
Actual	The negative value is passed to the <code>Event</code> constructor, which throws <code>IllegalArgumentException: Event time must be non-negative</code> .

Defect 2 — Missing validation on remaining burst time setter

Field	Detail
Class	Process
Method	setRemainingBurstTime
Description	The setter accepted negative values without any validation. A process's remaining burst time represents how many time units are left in the current CPU burst — a negative value is an impossible state in a real scheduler.
Expected	The setter throws <code>IllegalArgumentException</code> when a negative value is passed.
Actual	Silently accepted any negative value.

Defect 3 — Silent overwrite on processor assignment

Field	Detail
Class	Processor
Method	setCurrentProcess
Description	Assigning a non-null process to a processor that already has a non-null <code>currentProcess</code> silently overwrites the previous process reference with no error. The previously running process is lost.
Expected	The setter throws

Field	Detail
	<code>IllegalStateException</code> when a process is assigned to a non-free processor.
Actual	Silently overwrites the previous reference. Note: a higher-level guard in <code>EventDrivenSimulationEngine.dispatchProcessOnProcessor</code> calls <code>isFree()</code> before <code>setCurrentProcess</code> , so this state cannot be reached in normal execution — but the setter itself does not enforce the contract.

5. Assertions (Phase 3)

5.1 Assertion Strategy

Java's built-in `assert` statements were used to encode preconditions, postconditions, and class invariants directly in the production code. Assertions are active when the JVM is launched with the `-ea` flag and have no performance cost in production (without `-ea`).

The distinction between assertions and validation exceptions is intentional:

Mechanism	When thrown	Purpose	Example
<code>throw new IllegalArgumentException</code>	Always, regardless of JVM flags	Invalid input from an external source (user, file)	JSON config with negative memory size
<code>assert + AssertionError</code>	Only when <code>-ea</code> is active	Internal programming error — state that should never occur if the code is correct	Attempting to schedule a process on a non-free processor

This distinction separates user-facing error handling from developer-facing contract enforcement.

5.2 Assertions by Component

Process — preconditions and invariants on state transitions:

```
// Precondition on setState: state must not be null
assert state != null : "Process state must not be null";

// Precondition on setRemainingBurstTime: remaining time must be non-negative
assert remainingBurstTime >= 0 : "Remaining burst time must be non-negative, got: " + remainingBurstTime;

// Precondition on setLastProcessorId: ID must be non-negative
assert lastProcessorId >= 0 : "Last processor ID must be non-negative, got: " + lastProcessorId;

// Precondition on getCurrentSyscallDuration: syscall must follow the current
```

```

burst
assert hasSyscallAfterCurrentBurst() : "No syscall after current burst index " +
currentBurstIndex;

// Postcondition on advanceBurst: burst index advances by exactly 1
assert currentBurstIndex == previousIndex + 1 : "Burst index must advance by
exactly 1";

// Postcondition on advanceBurst: remaining time set to a positive value
assert remainingBurstTime > 0 : "Remaining burst time must be positive after
advancing";

```

Processor — invariants on process assignment:

```

// Precondition on setCurrentProcess: cannot assign to a busy processor
assert process == null || this.currentProcess == null
    : "Cannot assign process to a busy processor without releasing first";

// Class invariant: user process and system process cannot coexist
assert !(currentProcess != null && busyWithSystemProcess)
    : "Processor " + id + " cannot run a user process and a system process
simultaneously";

```

MemoryManager — preconditions, postconditions, and class invariant:

```

// Precondition on reserveSpace: amount must be positive and feasible
assert amount > 0 : "Reserve amount must be positive, got: " + amount;
assert getFreeMemory() >= amount : "Not enough free memory to reserve...";

// Postcondition on reserveSpace: reserved memory stays non-negative
assert reservedMemory >= 0 : "Reserved memory must not be negative after
reservation";

// Precondition on commitLoad: process must not already be loaded
assert !isLoading(process) : "Process " + process.getName() + " is already loaded
in memory";

// Postcondition on commitLoad: process must appear in loaded map
assert isLoading(process) : "Process must be loaded after commitLoad";

// Postcondition on unloadProcess: process must no longer be loaded
assert !isLoading(process) : "Process must not be loaded after unloadProcess";

// Class invariant: memory accounting must never exceed total
assert usedMemory >= 0 : "Used memory must not be negative";
assert reservedMemory >= 0 : "Reserved memory must not be negative";
assert usedMemory + reservedMemory <= totalMemory
    : "Used (" + usedMemory + ") + reserved (" + reservedMemory + ") exceeds
total memory (" + totalMemory + ")";

```

DiskController — preconditions and postconditions on disk operations:

```

// Preconditions on initiateMemoryLoad
assert process != null : "Process must not be null";
assert currentTime >= 0 : "Current time must be non-negative";
assert logEntries != null : "Log entries list must not be null";
assert eventQueue != null : "Event queue must not be null";
assert !memoryManager.isLoading(process) : "Process is already loaded in memory";

// Postcondition: disk must be busy at a future time after the operation
assert diskBusyUntil > diskBusyUntilBefore || diskBusyUntil > currentTime
    : "Disk must be busy after initiating a load";
assert loadEndTime > currentTime : "Load end time must be after current time";

```

Scheduler — preconditions and postconditions on scheduling decisions:

```
// Precondition on addToReadyQueue
assert process != null : "Cannot add null process to ready queue";

// Preconditions on scheduleReadyProcesses
assert processors != null : "Processors list must not be null";
assert memoryManager != null : "MemoryManager must not be null";

// Postcondition on dequeueForLoad: process removed from ready queue
assert !readyQueue.contains(toLoad) : "Process must be removed from ready queue after dequeue";

// Postcondition on findBestProcessor: returned processor must be free
assert result == null || result.isFree() : "Returned processor must be free";

// Postcondition on findFreeProcessor: every returned processor is free
assert processor.isFree() : "Returned processor must be free";
```

MinHeapPriorityQueue — invariants after every structural modification:

```
// Postcondition after add: size increases by 1
assert heap.size() == sizeBefore + 1 : "Heap size must increase by 1 after add";
// Heap property restored after add
assert checkHeapInvariant() : "Heap invariant violated after add";

// Postcondition after poll: size decreases by 1
assert heap.size() == sizeBefore - 1 : "Heap size must decrease by 1 after poll";
// Heap property restored after poll
assert checkHeapInvariant() : "Heap invariant violated after poll";

// Loop invariant inside checkHeapInvariant: each node ≤ its children
assert i == 0 || heap.get(i).compareTo(heap.get((i - 1) / 2)) >= 0
    : "Heap property violated at index " + i;
assert (2 * i + 1 >= size || heap.get(i).compareTo(heap.get(2 * i + 1)) <= 0)
    : "Heap property violated between parent " + i + " and left child";
assert (2 * i + 2 >= size || heap.get(i).compareTo(heap.get(2 * i + 2)) <= 0)
    : "Heap property violated between parent " + i + " and right child";
```

5.3 Relationship to Defects Found

Assertions provided a direct resolution path for two of the three defects found during unit testing:

- **Defect 2** (negative remaining burst time) → assertion added to `Process.setRemainingBurstTime`: `assert remainingBurstTime >= 0`. The `AssertionError` is now thrown immediately at the source of the invalid assignment, rather than propagating into a silent incorrect simulation state.
- **Defect 3** (silent processor overwrite) → assertion added to `Processor.setCurrentProcess`: `assert process == null || this.currentProcess == null`. Any attempt to assign a process to a non-free processor now immediately fails with an `AssertionError` in development builds.

For **Defect 1** (integer overflow in system process scheduling), the fix is at the validation layer (`IllegalArgumentException` in the `Event` constructor already catches the negative time) — an `assert` would be redundant here since the constraint is already enforced unconditionally.

Unit tests for the affected methods were also updated to verify that `AssertionError` is thrown for the impossible states confirmed by these two bugs.

6. Team Member Contributions

Team Member	Phase 1 — Implementation	Phase 2 — Unit Testing	Phase 3 — Assertions	Phase 4 — Documentation
Raul Tiron	Yes	Yes	—	—
Teodora Vlad	Yes	—	Yes	—
Cosmin Stan	Yes	—	—	Yes

Phase 1 — Application Development: All three team members contributed to the design and implementation of the simulator. The core simulation engine, scheduling algorithms, memory management, disk I/O model, input parsing, text output, and Gantt chart visualization were developed collaboratively.

Phase 2 — Unit Testing: Raul Tiron was responsible for the entire unit testing phase: setting up the testing infrastructure (JUnit 5, Mockito, JaCoCo in `pom.xml`), writing all 177 unit tests across the 10 test classes, performing the dependency injection refactoring required to enable isolated testing, and documenting the three defects found (`defects-found.md`).

Phase 3 — Assertions: Teodora Vlad was responsible for the entire assertions phase: introducing all `assert` statements across the codebase (`EventDrivenSimulationEngine`, `DiskController`, `MemoryManager`, `Process`, `Processor`, `ReadyQueue`, `Scheduler`, `MinHeapPriorityQueue`) and updating the affected unit tests to verify that `AssertionError` is thrown for impossible states.

Phase 4 — Documentation: Cosmin Stan was responsible for writing the final project documentation covering all three phases.